

cz88gfx

Graphics Library User Guide

The graphics library for the CADzilla S-100 graphics board (Hitachi HD63484 ACRTC and Brooktree Bt453 RAMDAC), for 8080, 8085 and Z80 programs written in assembly or C and built on the host with z88dk

Library version 1.1 · document revision 1.0 · 2026-10-11

Contents

1. Introduction **4**

1.1 What cz88gfx is . . . 4

1.2 One library, two languages . . . 4

1.3 The board, in brief . . . 5

1.4 About this guide . . . 5

2. Getting started **6**

2.1 A project that uses cz88gfx . . . 6

2.2 A first program . . . 6

3. Conventions **7**

3.1 Coordinates and colors . . . 7

3.2 In assembly . . . 7

3.3 In C . . . 8

3.4 Waiting, and errors . . . 8

4. Setting up the display **9**

4.1 Setup and the display modes . . . 9

4.2 Clearing . . . 9

4.3 Colors and the palette . . . 9

5. Lines, rectangles and polygons **10**

5.1 The drawing pointer . . . 10

5.2 Polylines and polygons . . . 11

6. Circles, ellipses and arcs **12**

7. Drawing modes **14**

7.1 Operation modes . . . 14

7.2 Color modes . . . 15

7.3 Area modes . . . 15

8. The pattern RAM **16**

8.1 Layout and the pattern pointer . . . 16

8.2 Loading patterns . . . 16

8.3 Line styles . . . 17

8.4 Drawing the pattern itself . . . 17

9. PAINT, flood and copy **18**

9.1 Painting areas . . . 18

9.2 Copying blocks 18

10. The overlay **20**

11. Text **21**

11.1 Fonts 21

11.2 Drawing text 23

12. Reading back **24**

13. Library versions **25**

13.1 Moving a program to another release 25

13.2 Making a release 25

Appendix A. Macro reference (assembly) **26**

Appendix B. Modules and routines **28**

Appendix C. C function reference **29**

Appendix D. Equates and constants **32**

Appendix E. Building and extending the library **33**

E.1 Adding to the library 33

E.2 How the C stubs work 33

Appendix F. Porting from czgfx **33**

Appendix G. Limits **34**

1. Introduction

cz88gfx is the graphics library for CADzilla programs built on the host with **z88dk**, in either of two languages: assembly, assembled by z88dk's z80asm, and **C**, compiled by z88dk's zcc. It sets up the board and gives a program the whole of the ACRTC's drawing processor: lines, rectangles, polygons, circles, ellipses and arcs, area fills and copies, the pattern RAM, the operation, color and area modes, a one-bit overlay plane and text in two fonts. The same library serves the 8080, 8085 and Z80.

This guide is for programmers using the library from either language. It explains the board's model as far as a program needs it, then each group of calls, showing the assembly and the C form side by side, with the screens of the demo programs as worked examples. The appendices give each language its own reference and explain how the library is built and extended.

1.1 What cz88gfx is

cz88gfx comes in three parts:

- `CZ88GFX.INC`, the header an assembly program INCLUDEs: the board's equates, an EXTERN for every library symbol, and the `G...` macros.
- `cz88gfx.h`, the header a C program #includes: a prototype for every C function, and the `CZ_` constants.
- `CZ88GFX.LIB`, a z80asm library: fourteen graphics modules and, from release 1.1, fifteen small modules of C bindings. The linker takes only the modules a program uses, so a program that draws only boxes gets no curve, text or font code, and an assembly program never carries the C bindings (Appendix B lists the modules and their sizes).

The library is 8080 code, in Intel mnemonics. z80asm links it unchanged into an 8080, 8085 or Z80 program, so one library serves every CPU a project can pick. It is linked on the host: nothing is mounted or copied in CP/M. Each project pins a numbered release, so a new cz88gfx version reaches a program only when the program moves to it (chapter 13).

cz88gfx is a port of **czgfx**, the library for programs built with Microsoft M80/L80 inside CP/M (`docs/czgfx-guide.pdf`). The routines, names, register conventions and macros are the same, and its graphics modules assemble to the same bytes as czgfx 1.0's. The two libraries share no files: a change to one doesn't reach the other.

1.2 One library, two languages

The C functions are not a second implementation. Each is a few instructions of assembly that take the C arguments off the stack and jump into the routine an assembly program calls. A C program therefore draws exactly what the assembly one does, at the same speed: **cgfxdemo** (C) draws **z88demo**'s (assembly) ten screens identically, pixel for pixel, and both match czgfx's gfxdemo.

Why not drive the board from C directly? The board's ports are available to C as variables (`<cadzilla.h>`), but sccz80, z88dk's C compiler for these CPUs, widens every port test to 16 bits: waiting for room in the ACRTC's FIFO costs 9 instructions and two library calls per pass in C, and 3 instructions in the library. A drawing library written in C would be bigger and slower than one it can simply call.

Language	Build	Header	Example
Assembly (`.Z88`)	`tools/z88dk/z88build.sh`	`INCLUDE "CZ88GFX.INC"`	`software/z88dk/z88demo`
C (`.c`)	`tools/z88dk/zccbuild.sh`	`#include <cz88gfx.h>`	`software/zcc/cgfxdemo`

1.3 The board, in brief

CADzilla pairs two chips of the mid 1980s. The **HD63484 ACRTC** (Advanced CRT Controller) generates the display and has a microcoded drawing processor: the host writes 16-bit commands and their parameters into an 8-word FIFO, and the chip draws into its own 2 MB of frame memory, which the CPU never addresses directly. The **Bt453** RAMDAC turns each 8-bit pixel into a color through its 256-entry palette. A separate one-bit **overlay** plane can be shown over the picture in a color of its own. The board decodes eight I/O ports from `70H`:

Port	Assembly	C (`<cadzilla.h>`)	Use
70H	`ACRSR`	`CZ_ACRTC_SR / _AR`	ACRTC status (in) / address register (out)
71H	`MODEREG`	`CZ_MODE`	the board's MODE register: sync polarity, access mode, overlay enable and select (write-only)
72H	`ACRDAT`	`CZ_ACRTC_DATA`	ACRTC control register data / the command FIFO
74H	`DACADR`	`CZ_DAC_ADDR`	Bt453 address register
75H	`DACPAL`	`CZ_DAC_PAL`	Bt453 palette RAM (R, G, B)
77H	`DACOVL`	`CZ_DAC_OVL`	Bt453 overlay colors (R, G, B)

cz88gfx programs the board for 8 bits a pixel, one screen, at 640x480, 800x600 or 1024x768. A program never needs the ports itself, but they are there if it does. The library is assembled for the board at I/O base `70H`.

1.4 About this guide

It describes cz88gfx 1.1. Macro, routine and function names, constants, commands and source are set in `Courier`. "CP" is the ACRTC's current drawing pointer. Paths are relative to the CADzilla repository. The library's own reference, kept with the sources, is `software/z88dk/cz88gfx/README.md`; the toolchain itself is described in `tools/z88dk/README.md`.

2. Getting started

2.1 A project that uses cz88gfx

CADzilla programs run under CP/M 2.2 on the altairsim emulator (or on the real board), each project with its own machine and disks in its `work/` directory. A z88dk project is built on the host; CP/M only receives the finished `.COM`. A new project, assembly or C, is scaffolded with:

```
.claude/skills/new-cadzilla-project/scripts/new_project.sh myprog \
        --toolchain z88dk --cpu z80 --mode 1024x768     # assembly
.claude/skills/new-cadzilla-project/scripts/new_project.sh myprog \
        --toolchain zcc --cpu z80 --mode 1024x768       # C
```

That creates, under `software/z88dk/myprog/` (or, for C, `software/zcc/myprog/`):

File	What it is
`src/MYPROG.Z88` or `src/myprog.c`	a starter program. C sources are lowercase (zcc takes only a lowercase `.c`); everything else is uppercase
`machine.toml`	the monitor resolution the project is strapped for, and the CPU, 8080, 8085 or Z80 (tracked). The build and the emulator both follow it
`cz88gfx.version`	the cz88gfx release the project builds against, e.g. `1.1` (tracked)
`build/`	the build's output: `MYPROG.COM`, its map, listings and symbols (gitignored)
`work/`	its altairsim machine and CP/M disks (gitignored)

2.2 A first program

The same complete program in each language: it sets up the board, draws a frame round the screen and a blue box in the middle, and waits for the drawing to finish.

Assembly (z80asm)

```
        INCLUDE "CZ88GFX.INC"
        ORG     0100H
START:  GINIT   CZM1K
        GCOLOR  WHITE+BRIGHT
        GBOX    0,0,1023,767
        GFBOX   448,320,576,448,BLUE+BRIGHT
        GWAIT
        RET                     ;to CP/M
```

C (zcc)

```
#include <cz88gfx.h>

int main(void)
{
    cz_init(CZ_M1K);
    cz_color(CZ_WHITE + CZ_BRIGHT);
    cz_box(0, 0, 1023, 767);
    cz_fbox(448, 320, 576, 448,
            CZ_BLUE + CZ_BRIGHT);
    cz_wait();
    return 0;
}
```

It is built on the host, against the release the project pins:

```
$ tools/z88dk/z88build.sh software/z88dk/myprog      # assembly
z88dk: MYPROG for z80 (MYPROG.Z88), cz88gfx 1.1
$ tools/z88dk/zccbuild.sh software/zcc/myprog        # C
zcc: MYPROG for z80 (myprog.c), cz88gfx 1.1
```

Either way the result is `build/MYPROG.COM`. In altairsim, point the host bridge at `build/`, copy it onto the project's disk with `R` and run it:

```
A0>R MYPROG.COM
A0>MYPROG
```

`build/MYPROG.MAP` shows which library modules went in, and `build/MYPROG.SYM` holds the symbols for altairsim's `SYMBOLS LOAD`. The `cadzilla-build-z88dk` and `cadzilla-build-zcc` skills drive the whole cycle.

3. Conventions

3.1 Coordinates and colors

The library puts the drawing origin at the **bottom-left** pixel. X runs from 0 to `CZXMAX` left to right, Y from 0 to `CZYMAX` bottom to top. These are words in the library that setup fills in for the mode (1023 and 767 at 1024x768): `LHLD CZXMAX` in assembly, `cz_xmax` in C. A color is a palette index, 0-255 (chapter 4).

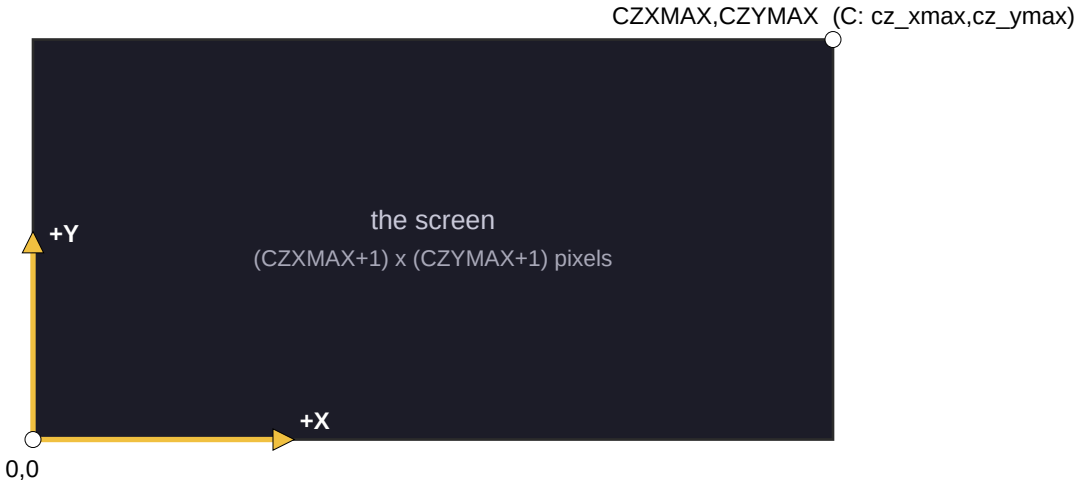


Figure 1. Screen coordinates.

3.2 In assembly

- X goes in HL, Y in DE, a color in A. Every routine preserves BC, DE and HL unless its entry says otherwise, and clobbers A and the flags. A few macros clobber registers of their own; Appendix A says which.
- **Every macro argument is required.** z80asm's macros can't test for a blank argument. To pass a computed value, load the registers and call the routine the macro names: `LXI D,420` / `CALL CZMOVE`. To draw round the CP, `LXI H,7` / `CALL CZCRCL`. A default is written out: `GMODE 0`, `GTATTR 2,0,0`.
- A list is one argument in parentheses: `GPTS TRI,(480,448,576,704,400,640)`. When the list is the macro's only argument (`GPLINE`, `GPOLY`), double the parentheses: `GPOLY ((400,24,488,24,444,184))`. An argument that starts with "(" must end with its ")": write `0+(N % 8)*32`, not `(N % 8)*32+1`.
- Strings are double-quoted: `GTEXT "Hello"`; single quotes are for one character.
- Library names are plain: the header declares them all EXTERN, so `CALL CZLDW` and `LHLD CZXMAX` just work, and an EXTERN nothing uses links nothing in. Every library global is `CZ` plus up to four characters; the header also defines the hue constants, `OP...`, `OM...`, `CM...`, `AM...`, `CP...`, the port names and `SR...` (Appendix D). Don't reuse these names; z80asm is case-sensitive.
- **z80asm traps.** `JP x` and `CP x` are Zilog's unconditional jump and compare, not Intel's jump/call if plus (the build warns). An assembly-time counter is `DEFL`, and must be parenthesized, `N DEFL (N+1)`:

z80asm substitutes its text, so after two `N DEFL N+1`, `N*19` is 0+1+1*19.

3.3 In C

- Coordinates are `int` (relative moves can be negative); colors, modes and other byte values `unsigned char`; sizes and radii `unsigned int`.
- Each macro is a function named `cz_` plus its verb: `GCOLOR` is `cz_color`, `GBOX` `cz_box`, `GOVL` `cz_ovl`; `GMODE` is `cz_setmode`. Where assembly calls a routine "at the CP", C has a `_cp` function: `cz_circle_cp(r)`. Appendix C lists them all.
- Constants carry `CZ_`: `CZ_RED + CZ_BRIGHT`, `CZ_OM_EOR`, `CZ_CM_CL1`, `CZ_AM_CLIP`, `CZ_CP_R90`, `CZ_DIR_CW`. `cz88gfx.h` includes `<cadzilla.h>`, which adds the ports and the status bits `CZ_SR_CER`, `CZ_SR_ARD` and so on.
- Point and pattern tables are C arrays in the library's own layout, the count first: `static const int tri[] = { 3, 480,448, 576,704, 400,640 };`.
- Calling conventions (sccz80): a one-argument function is `__z88dk_fastcall`, its argument in HL; one with more is `__smallc __z88dk_callee`, which pushes the arguments left to right and leaves the function to pop them, so a call site has no clean-up code. The header declares this; a program just calls. Nothing uses the Z80's IX or IY.
- Don't define names of your own beginning `cz_` or `CZ_`. The library never calls the BDOS; console I/O is the program's.

3.4 Waiting, and errors

Drawing calls only send commands; the ACRTC draws them from its FIFO while the program goes on. `GWAIT` / `cz_wait()` waits until the FIFO is empty and the last command has ended, and returns the status register: in assembly in A, with **Z** if the drawing ended cleanly and **NZ** on a command error; in C as the return value, where `cz_wait() & CZ_SR_CER` tests for the error (CER, status bit 7: an undefined command or a bad parameter). `GABORT` / `cz_abort()` stops the chip, flushes the FIFOs and clears CER. On the real board, and in altairsim with `draw_rate = "real"`, drawing takes time, and waiting is also how a program knows it has finished.

4. Setting up the display

4.1 Setup and the display modes

`GINIT mode` / `cz_init(mode)` sets up the whole board for one display mode: the MODE register, the ACRTC's timing and display registers, the default palette, the drawing parameters, the overlay, and a clear screen. The mode is a descriptor in the library:

Assembly	C	Mode	Screen
`CZM640`	`CZ_M640`	VESA 640x480 @ 60 Hz	X 0-639, Y 0-479
`CZM800`	`CZ_M800`	VESA 800x600 @ 60 Hz	X 0-799, Y 0-599
`CZM1K`	`CZ_M1K`	VESA 1024x768 @ 60 Hz	X 0-1023, Y 0-767

The monitor is fixed-frequency, chosen by a strap (in altairsim, cad0's `mode` in the project's `machine.toml`; 1024x768 by default). The descriptor must match it: the board doesn't reject a mismatch, the picture just lands in the wrong place in the monitor's frame (altairsim's `board_get cad0` shows the picture's position). Setup clobbers all registers; a descriptor of your own goes in HL for `CALL CZINIT`, or to `cz_init` as a pointer.

After setup: the screen is color 0; the drawing color (CL1) is 255, white; CL0 and CCMP are 0; the drawing mode is replace with no area checking; the pattern RAM is all ones; the area is the whole screen; the overlay is clear, hidden and white; and drawing goes to the frame.

4.2 Clearing

`GCLS c` / `cz_cls(c)` clears the whole screen to color c with the ACRTC's block clear, the fastest way to fill the screen. `GOVCLS` / `cz_ovcls()` clears the overlay (chapter 10).

4.3 Colors and the palette

A pixel is a palette index, 0-255. `GCOLOR c` / `cz_color(c)` sets the drawing color. Setup loads a default palette of eight hues with 32 brightness levels each: **index = hue + level**, where the hue is one of the constants below and the level runs from 0 (black) to `BRIGHT` / `CZ_BRIGHT` (31). `CYAN+BRIGHT` / `CZ_CYAN + CZ_BRIGHT` is full cyan; `RED+16` / `CZ_RED + 16` a half-bright red.

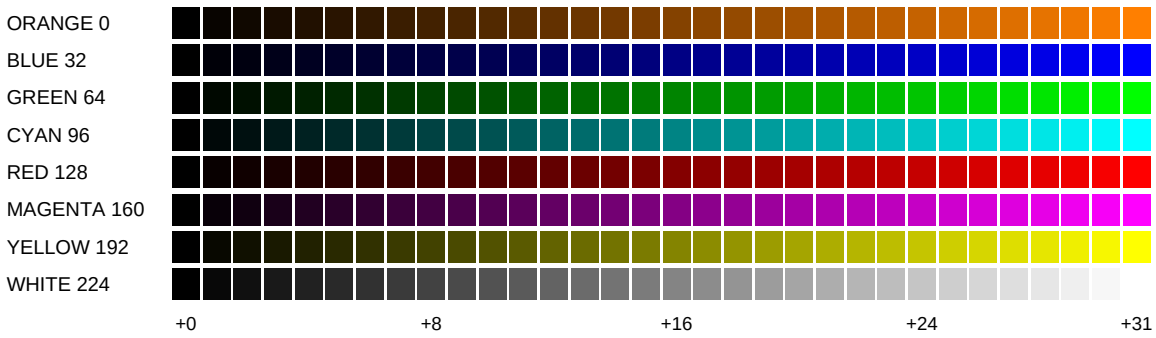
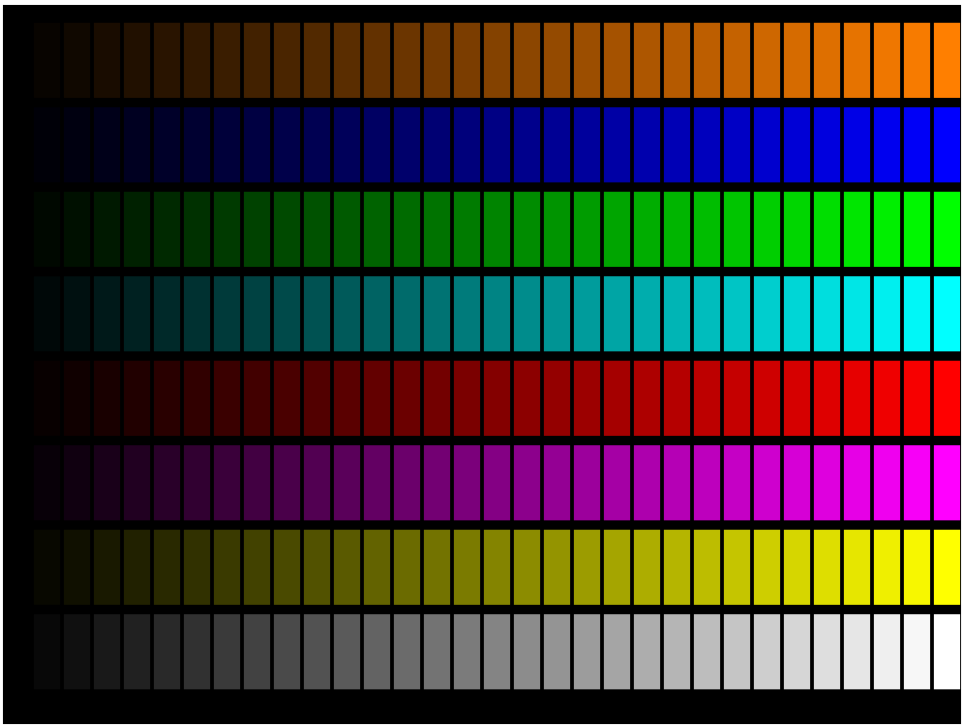


Figure 2. The default palette, as setup loads it. In C the hues are CZ_ORANGE ... CZ_WHITE.

`GPAL i,r,g,b` / `cz_pal(i, r, g, b)` sets palette entry i to any color (0-255 each component; the macro clobbers BC and E). Changing an entry recolors everything already drawn in it at once, which makes palette animation cheap. `CALL CZPALD` / `cz_paldef()` loads the default palette again.



Demo screen 1: all 256 entries of the default palette as filled boxes, in a white frame (GFRECT, GBOX / cz_frect, cz_box).

5. Lines, rectangles and polygons

5.1 The drawing pointer

The ACRTC draws from its current drawing pointer, the CP. A move sets it; the drawing calls draw from it and most leave it at their end point. Each comes in an absolute form, to x,y, and a relative one, by dx,dy from the CP:

Assembly	C	Draws	CP afterwards
`GMOVE x,y` `GRMOVE dx,dy`	`cz_move(x, y)` `cz_rmove(dx, dy)`	nothing: moves the CP	x,y
`GLINE x,y` `GRLINE dx,dy`	`cz_line(x, y)` `cz_rline(dx, dy)`	a line from the CP, its end point **not** drawn	x,y
`GRECT x,y` `GRRECT dx,dy`	`cz_rect(x, y)` `cz_rrect(dx, dy)`	a rectangle outline, corners CP and x,y	unchanged
`GFRECT x,y` `GRFRCT dx,dy`	`cz_frect(x, y)` `cz_rfrect(dx, dy)`	a filled rectangle, both corners included	one row past the far edge
`GDOT x,y`	`cz_dot(x, y)`	one pixel	x,y

Because a line's end point isn't drawn, a closed shape drawn from lines lights each corner exactly once, which matters when drawing in EOR mode. In assembly, with X and Y already in HL and DE, call the routine directly: `CZMOVE`, `CZLINE`, `CZRECT`, `CZFRCT`, `CZDOT` and the relative `CZRMOV`, `CZRLIN`, `CZRRCT`, `CZRFRC`. Three calls combine a move with a drawing:

Assembly	C	Does
`GSEG x1,y1,x2,y2`	`cz_seg(x1, y1, x2, y2)`	a line from x1,y1 to x2,y2 (x2,y2 not drawn)
`GBOX x1,y1,x2,y2`	`cz_box(x1, y1, x2, y2)`	a rectangle outline with corners x1,y1 and x2,y2
`GFBOX x1,y1,x2,y2,c`	`cz_fbox(x1, y1, x2, y2, c)`	a filled rectangle in color c (which becomes the drawing color)

5.2 Polylines and polygons

A polyline is drawn through a list of points; a polygon is closed back to its first point. The library takes a **point table**: the number of points, then the X,Y pairs, at least 2 points.

Assembly (z80asm)

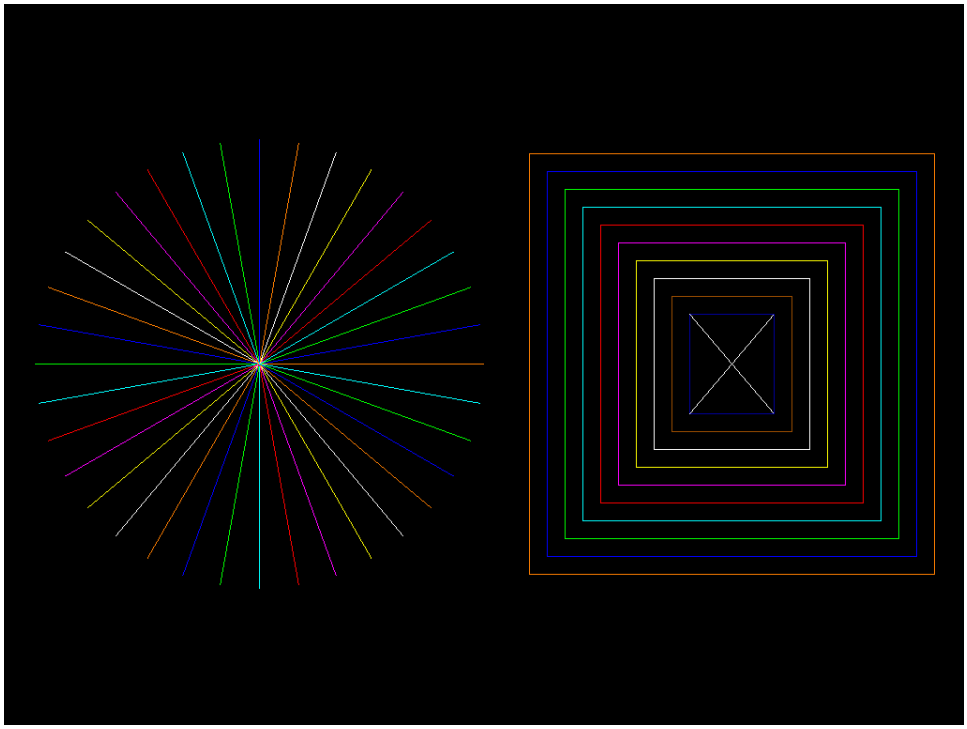
```
        GPOLY   ((240,704,202,595,88,594))
        GPLG    TRI     ;from a table
        GPLT    SINE    ;open, from a table
        ...
        GPTS    TRI,(480,448,576,704,400,640)
SINE:   DW      33      ;by hand: the count,
        DW      64,152,93,186,...  ;the pairs
```

C (zcc)

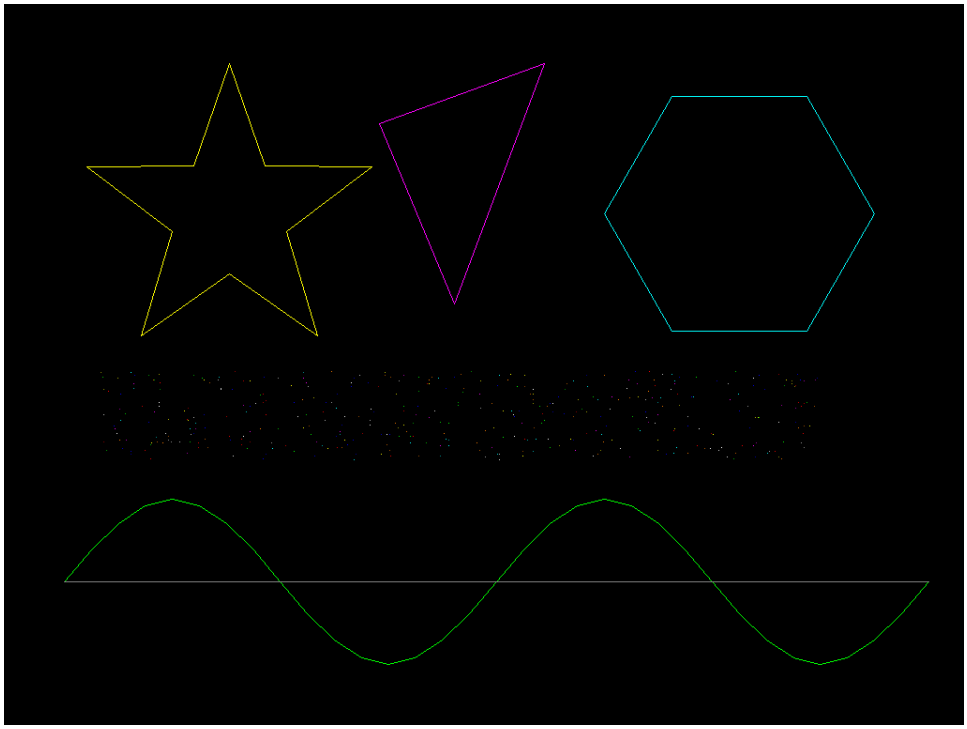
```
static const int tri[] =
    { 3, 480,448, 576,704, 400,640 };
static const int sine[] =
    { 33, 64,152, 93,186, ... };

cz_polygon(tri);
cz_polyline(sine);
```

In assembly `GPLINE` and `GPOLY` place their points inline (and jump over them); `GPLT tbl` and `GPLG tbl` draw from a table, which `GPTS` builds. A macro call must fit on one line, so write a longer table's `DW`s by hand in the same layout. These macros clobber HL. In C, `cz_polyline(t)` and `cz_polygon(t)` take the array.



Demo screen 2: a 36-spoke starburst from a data table (lines from the centre), and ten nested rectangles from a loop (GBOX / cz_box).



Demo screen 3: a star, a hexagon and a triangle (polygons), a sine wave (a polyline) and 400 random dots.

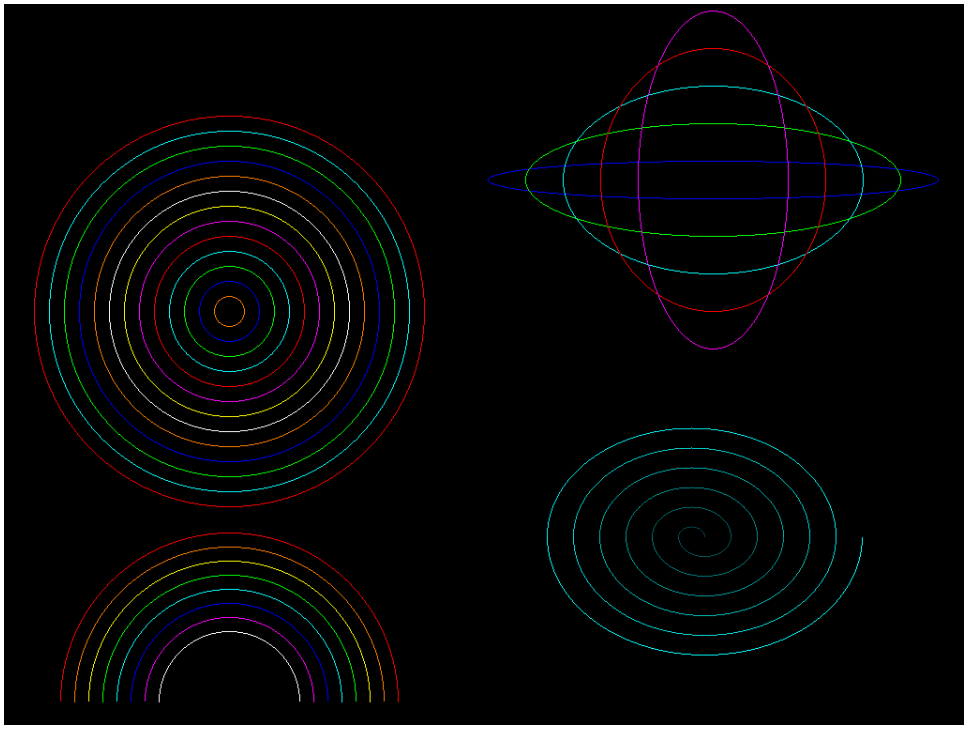
6. Circles, ellipses and arcs

Assembly	C	Draws
`GCIRC x,y,r`	`cz_circle(x, y, r)`	a circle of radius r centred on x,y; the CP is left at the centre
`CALL CZCRCL (HL = r)`	`cz_circle_cp(r)`	the same, round the CP
`GELLIP x,y,rx,ry`	`cz_ellipse(x, y, rx, ry)`	an ellipse with semi-axes rx (along X) and ry (along Y), each up to 4095
`CALL CZELPS (HL, DE)`	`cz_ellipse_cp(rx, ry)`	the same, round the CP
`GARC xc,yc,xe,ye,dir`	`cz_arc(xc, yc, xe, ye, dir)`	an arc from the CP round the centre xc,yc to where it meets the ray from the centre through xe,ye
`GRARC ...`	`cz_rarc(...)`	the same, the centre and xe,ye relative to the CP
`GEARC rx,ry,xc,yc,xe,ye,dir`	`cz_earc(rx, ry, xc, yc, xe, ye, dir)`	an arc of an ellipse whose semi-axes are in the ratio rx:ry
`GREARC ...`	`cz_rearc(...)`	the same, relative to the CP

- An arc starts at the CP, and its radius is the CP's distance from the centre. It runs counter-clockwise (`DIRCCW` / `CZ_DIR_CCW`) or clockwise (`DIRCW` / `CZ_DIR_CW`), and stops at the ray through xe,ye, so xe,ye need not be exactly on the curve. The CP is left at xe,ye, which is not drawn.
- For an elliptical arc only the ratio rx:ry matters (each at most 90): the size comes from the CP. Started on the ellipse's horizontal axis, a distance d from the centre, the arc lies on the ellipse with semi-axes d along

X and d * ry / rx along Y.

- The chip takes the squares of rx and ry. The macro computes them at assembly time; in C, `cz_earc` and `cz_rearc` are macros that square their arguments (at compile time for constants, and so evaluate each twice) for the functions `cz_earc_sq` and `cz_rearc_sq`.
- Circles and ellipses are drawn in order round the curve, so a dashed line style runs along them (chapter 8).
- The arc macros clobber HL.



Demo screen 4: concentric circles, a rainbow of half-circle arcs drawn both ways round, ellipses, and a spiral of elliptical half arcs.

7. Drawing modes

Every ACRTC drawing command carries a mode in its low byte. cz88gfx keeps that byte, `CZDM`, and adds it to every drawing command it sends (all but the moves). Setup resets it to 0: replace, two-color pattern, no area checking.

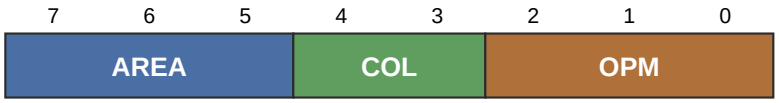


Figure 3. The drawing mode byte.

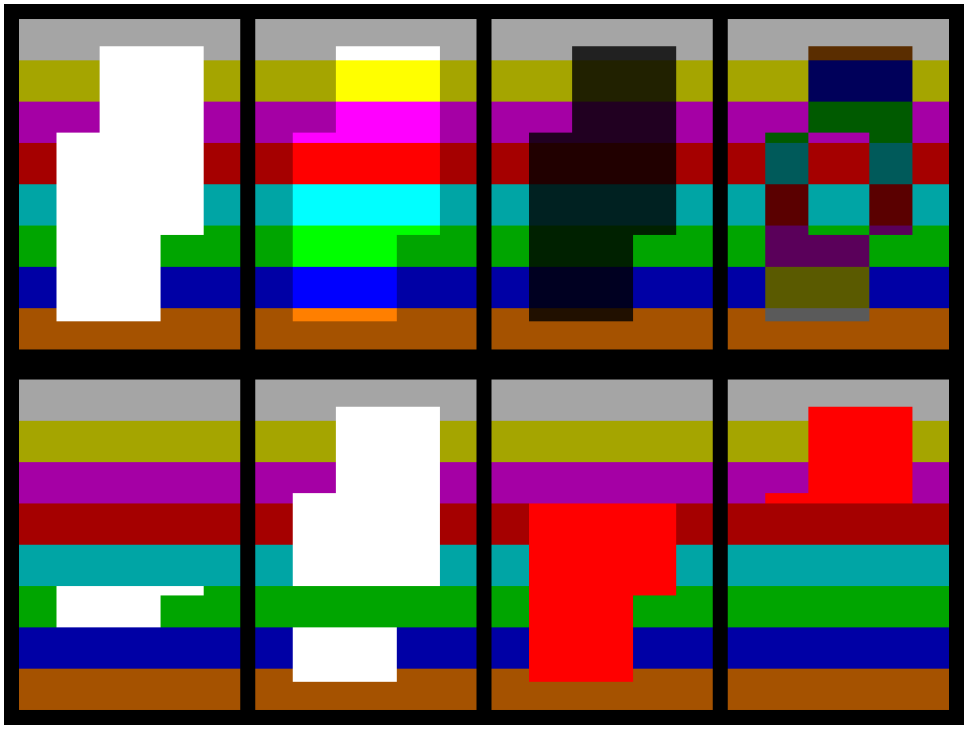
`GMODE m` / `cz_setmode(m)` sets the whole byte, e.g. `GMODE OMEOR+CMCL1+AMCLIP` / `cz_setmode(CZ_OM_EOR + CZ_CM_CL1 + CZ_AM_CLIP)`; 0 resets it. `GOPM`, `GCMODE` and `GAMODE` / `cz_opm`, `cz_cmode` and `cz_amode` each set one field and keep the others.

7.1 Operation modes

The operation mode says how the color C being drawn meets the pixel P already there:

Assembly	C	Result
`OMREP`	`CZ_OM_REP`	C (replace)
`OMOR`	`CZ_OM_OR`	P OR C
`OMAND`	`CZ_OM_AND`	P AND C
`OMEOR`	`CZ_OM_EOR`	P XOR C: drawing the same thing twice restores the picture
`OMEQ`	`CZ_OM_EQ`	C only where P = CCMP (GCCMP c / cz_ccmp(c))
`OMNE`	`CZ_OM_NE`	C only where P is not CCMP
`OMLT`	`CZ_OM_LT`	C only where P < C
`OMGT`	`CZ_OM_GT`	C only where P > C

The logic and comparisons work on the palette index.



Demo screen 5: in each panel, two overlapping boxes drawn over colored bands in one operation mode. Top: replace, OR, AND, EOR (where the boxes overlap, EOR restores the bands). Bottom: =CCMP, <>CCMP, C.

7.2 Color modes

The ACRTC draws every pixel through its pattern RAM (chapter 8). The color mode says what each pattern bit draws. With setup's all-ones pattern, every mode but CL0-only simply draws in CL1. CL0 is set with `GBCOL c` / `cz_bcol(c)`.

Assembly	C	A 1 bit draws	A 0 bit draws
`CMSEL`	`CZ_CM_SEL`	CL1 (the drawing color)	CL0
`CMCL1`	`CZ_CM_CL1`	CL1	nothing: the pattern is transparent
`CMCL0`	`CZ_CM_CL0`	nothing	CL0
`CMPRAM`	`CZ_CM_PRAM`	the pattern RAM is a 4x4 tile of colors	

7.3 Area modes

The area modes check each pixel against a rectangle, set with `GAREA x1,y1,x2,y2` / `cz_area(x1, y1, x2, y2)` (low and high corners, both inside) and reset to the whole screen with `GAFULL` / `cz_afull()`:

Assembly	C	Effect
`AMNONE`	`CZ_AM_NONE`	no checking
`AMCLIP`	`CZ_AM_CLIP`	draw only inside the area
`AMHOLE`	`CZ_AM_HOLE`	draw only outside it
`AMCLPD`	`CZ_AM_CLPD`	as AMCLIP, and set ARD if a pixel was dropped
`AMHOLD`	`CZ_AM_HOLD`	as AMHOLE, and set ARD if a pixel was dropped

Assembly	C	Effect
AMSTOP	CZ_AM_STOP	stop at the first pixel outside the area, and set ARD
AMSTPI	CZ_AM_STPI	stop at the first pixel inside the area, and set ARD

ARD is status bit 6. After waiting, `ANI SRARD` tests it in assembly, `cz_wait() & CZ_SR_ARD` in C. After a stop the CP is the pixel where the drawing stopped; `GGETCP` / `cz_getcp(&x, &y)` reads it back (chapter 12), and also clears ARD.

8. The pattern RAM

The ACRTC draws every pixel, of lines and fills alike, through its 16-word pattern RAM. Setup fills it with ones, so everything draws in CL1 until a program loads a pattern of its own.

8.1 Layout and the pattern pointer

- In the bit color modes word n is pattern **row n** and bit m its **column m**. Row 0 is the first row drawn (the bottom of a fill), bit 0 the leftmost pixel.
- A pattern pointer walks the RAM: one bit per pixel along a line (continuing round the corners of a rectangle, polygon or circle), one row per raster in a fill.
- `GPSCAN x0,y0,x1,y1,zx,zy` / `cz_pscan(x0, y0, x1, y1, zx, zy)` sets the window it walks, columns x0..x1 and rows y0..y1, and a zoom that draws each bit zx+1 times across and zy+1 times up (0,0: none). It also puts the pointer at the window's start.
- The pointer is **live**: it stays where the last drawing left it. Reset it with `GPPTR x,y` / `cz_pptr(x, y)` before a fill whose tiling must line up. `GPDEF` / `cz_pdef()` puts everything back as setup left it.

8.2 Loading patterns

A **pattern table** is the number of words, then the words. Loading one fills the pattern RAM from a given word (0: the start). A color tile for `CMPRAM` / `CZ_CM_PRAM` is a pattern table of 16 words, each a color in both bytes, its first four the bottom row.

Assembly (z80asm)

```
        GLDPAT  RING,0
        GPSCAN  0,0,15,15,0,0
        GCMODE  CMCL1   ;0 bits transparent
        GFBOX   100,100,300,200,YELLOW+BRIGHT
        ...
        GPAT    RING,(0,0,07E0H,1FF8H,...)
        GTILE   TILE,(159,223,95,127,...)
```

C (zcc)

```
static const unsigned int ring[] =
    { 16, 0, 0, 0x07E0, 0x1FF8, ... };
static const unsigned int tile[] =
    { 16, CZ_TILE(159), CZ_TILE(223), ... };

cz_ldpat(ring, 0);
cz_pscan(0, 0, 15, 15, 0, 0);
cz_cmode(CZ_CM_CL1);
cz_fbox(100, 100, 300, 200,
        CZ_YELLOW + CZ_BRIGHT);
```

In assembly `GPAT` and `GTILE` build the tables and `GLDPAT` clobbers HL. In C, `CZ_TILE(c)` makes a tile word from a color. `GPSCAN` takes constants; with the window in a 6-byte table at run time, `CALL CZPSCN`. `cz_pscan` takes run-time values.

8.3 Line styles

`GLSTYL pat,zoom` / `cz_lstyle(pat, zoom)` is the quick way to dashed lines: pattern row 0 becomes the 16-bit style (bit 0 first), the window becomes 16x1 and the pointer is reset. Fills use the style too, so set it back with `GLSTYL 0FFFFH,0` / `cz_lstyle(0xFFFF, 0)` (or the pattern defaults) afterwards. In `CMCL1` the gaps leave the picture alone; in `CMSEL` they draw in CL0.

Assembly (z80asm)	C (zcc)

`GCMODE CMCL1`
`GLSTYL 0F0F0H,0 ;4 on, 4 off`
`GBOX 40,24,200,184`
`GLSTYL 0FFFFH,0 ;solid`
	`cz_cmode(CZ_CM_CL1);`
	`cz_lstyle(0xF0F0, 0); /* 4 on, 4 off */`
	`cz_box(40, 24, 200, 184);`
	`cz_lstyle(0xFFFF, 0); /* solid */`

8.4 Drawing the pattern itself

`GPTN x,y,w,h,dir` / `cz_ptn(x, y, w, h, dir)` draws the pattern RAM as a w x h picture (1-256 each) with its corner at x,y. Its rows run in direction dir, 45 degrees apart counter-clockwise from +X, and are stacked 90 degrees further round; adding `PTSLNT` / `CZ_PTSLNT` stacks them only 45 degrees round, slanting the picture. A diagonal direction leaves gaps between pixels, as the chip's PTN does. Text (chapter 11) is drawn this way. `cz_ptn_cp(w, h, dir)` draws at the CP.

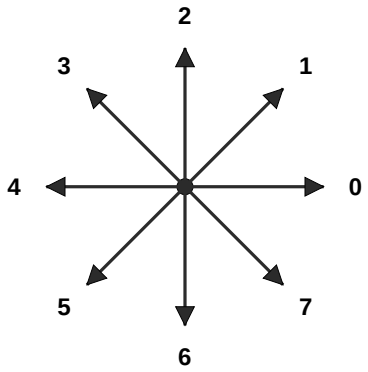
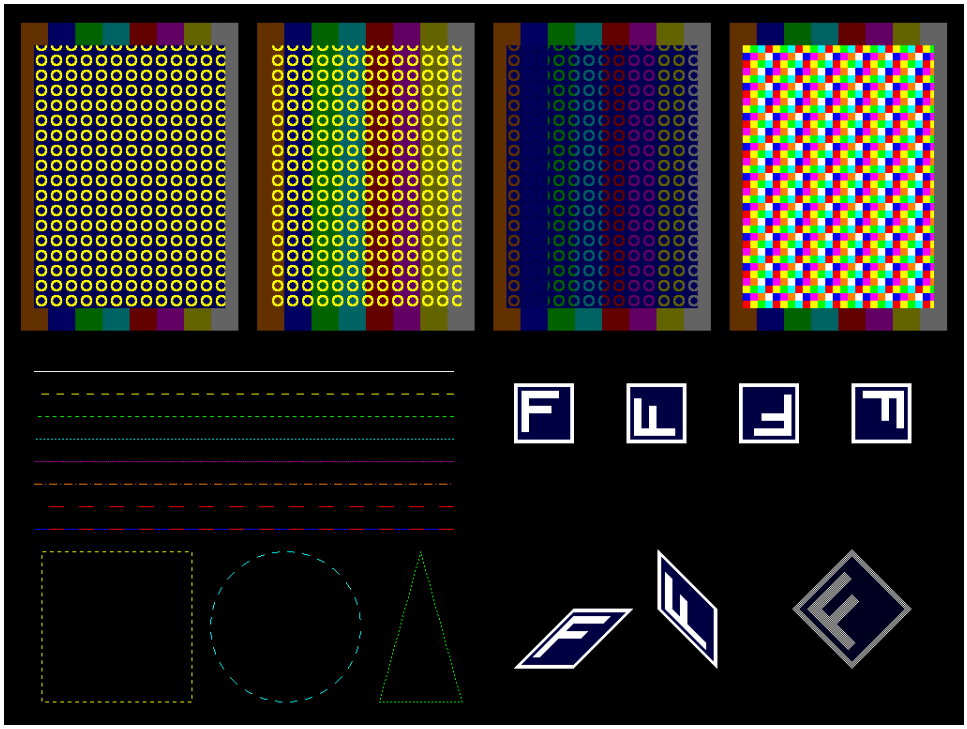


Figure 4. Directions 0-7, used by the pattern drawing and text.



Demo screen 6. Top: a ring pattern filled in CMSEL, CMCL1 and CMCL0, and a 4x4 color tile in CMPRAM. Bottom left: line styles, and dashes running round a box, a circle and a triangle. Bottom right: the pattern drawn turned 0, 90, 180 and 270 degrees, slanted, and at 45 degrees.

9. PAINT, flood and copy

9.1 Painting areas

`GPAINT x,y,e` / `cz_paint(x, y, e)` fills outward from x,y, in the current color, pattern and modes, up to pixels of the edge color e. `GFLOOD x,y,c` / `cz_flood(x, y, c)` instead fills the connected region of color c round x,y, recoloring one shape. From the CP: `CALL CZPANT` / `CZFLOD` with the color in A, `cz_paint_cp(e)` / `cz_flood_cp(c)`.

- **CL0 and CL1 are edges too.** The chip also stops at any pixel already in CL0 or CL1. After setup CL0 is 0, so painting over black paints nothing until CL0 is set to something else. For a solid fill, set it to the fill color.
- **Outlines must be closed.** The fill is 4-connected, so an 8-connected outline (lines, circles) holds it, but a line's end point isn't drawn: run spokes a pixel past a circle.
- **On the real board**, PAINT keeps only four pending seed points and hands any more to the host, which cz88gfx doesn't do, so a complicated area may be left partly unfilled. altairsim has no such limit.

9.2 Copying blocks

`GCOPY x1,y1,x2,y2,xd,yd,t` / `cz_copy(x1, y1, x2, y2, xd, yd, t)` copies the rectangle with corners x1,y1 and x2,y2 so that corner x1,y1 lands on xd,yd, transformed by t (`CPNORM` / `CZ_CP_NORM`: as it is). The operation and area modes apply at the destination; the color mode doesn't, since the source pixel is the color. The macro clobbers HL.

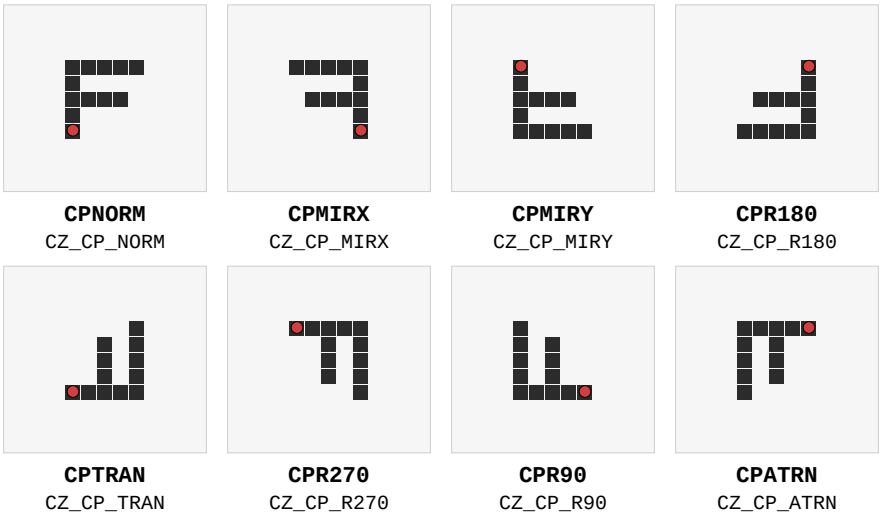
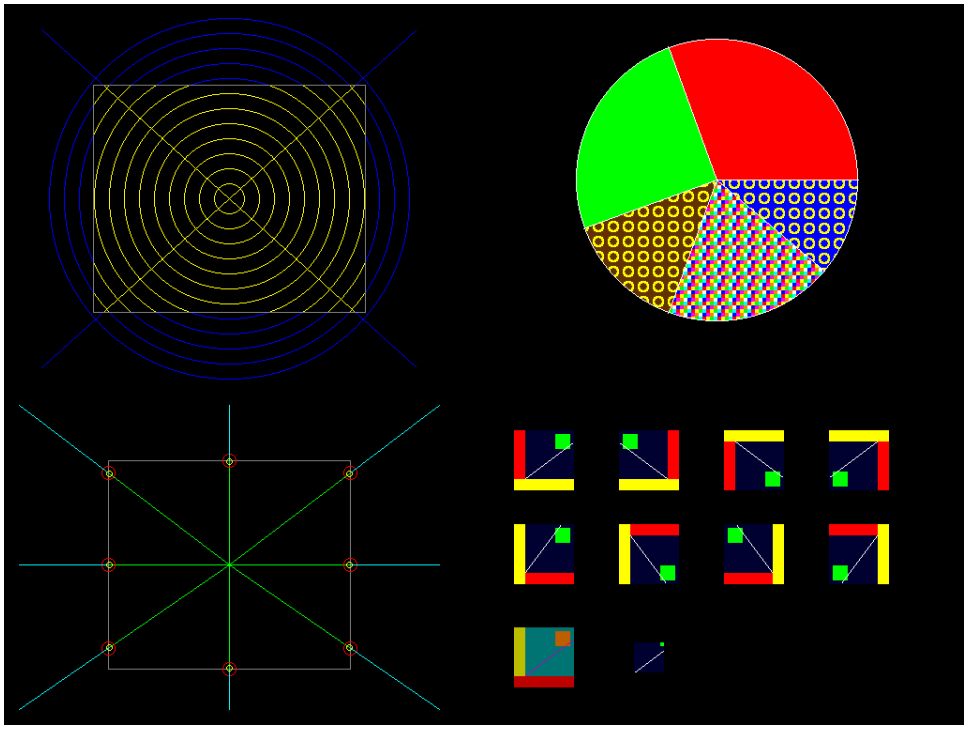


Figure 5. The copy's transforms, shown on an F. The red dot is the source corner x1,y1, which lands on xd,yd.



Demo screen 7. Top left: circles clipped to an area (AMCLPD, yellow) and to outside it (AMHOLD, blue). Bottom left: lines that stop at the edge of an area (AMSTOP, AMSTPI), ringed where the CP was read back. Top right: a pie chart painted solid, patterned and flood-recolored. Bottom right: a picture copied in all eight orientations, in EOR, and clipped.

10. The overlay

The board has a second plane over the 8-bit picture: the **overlay**, one bit per pixel in its own memory. Wherever a bit is set and the overlay is shown, the Bt453 shows the overlay color instead of the pixel's palette color; a clear bit lets the picture through. So what is drawn on the overlay can be shown, hidden, recolored or erased without disturbing the picture under it: a cursor, a grid, selection marks, rubber-band lines, text.

Assembly	C	Does
`GOVON / GOVOFF`	`cz_ovon() / cz_ovoff()`	show / hide the overlay; what is drawn in it stays
`GOVPAL r,g,b`	`cz_ovpal(r, g, b)`	the overlay color (the macro clobbers BC, E)
`GOVL`	`cz_ovl()`	draw on the overlay from here on
`GFRAME`	`cz_frame()`	draw on the frame again
`GOVCLS`	`cz_ovcls()`	clear the whole overlay, from either

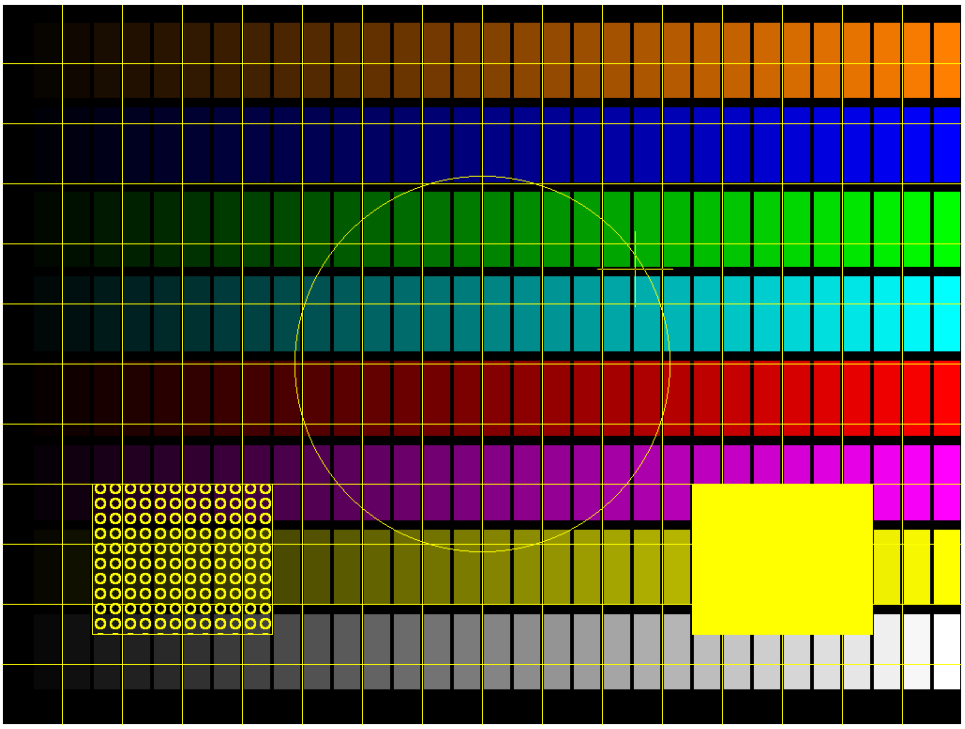
- Between the switch to the overlay and back, every drawing call draws on the overlay, with the frame's own coordinates.
- Overlay pixels are one bit, so the only colors are `OVSET` / `CZ_OVSET` (on) and `OVCLR` / `CZ_OVCLR` (off).
- EOR toggles bits: drawing a shape twice undraws it, the classic moving cursor.
- Patterns, the area modes and PAINT work on the overlay (set CL0 to `OVSET` before painting); color tiles mean nothing at one bit a pixel. The copy works within the overlay, not between the planes.
- Both switches wait for the ACRTC to finish, since they change its pixel depth, and leave the CP at 0,0: move afterwards.

Assembly (z80asm)

```
        GOVPAL  255,255,0       ;yellow
        GOVL
        GCOLOR  OVSET
        GMODE   OMEOR
        CALL    XHAIR   ;draw a crosshair
        CALL    XHAIR   ;undraw it
        GMODE   0
        GFRAME
        GOVON
```

C (zcc)

```
cz_ovpal(255, 255, 0);   /* yellow */
cz_ovl();
cz_color(CZ_OVSET);
cz_setmode(CZ_OM_EOR);
xhair();                 /* draw */
xhair();                 /* undraw */
cz_setmode(0);
cz_frame();
cz_ovon();
```



Demo screen 8: the palette of screen 1 on the frame, and on the overlay in yellow a grid, a circle, a patterned and a solid box, and a crosshair moved in EOR. Hiding the overlay shows screen 1 unchanged.

How it works: the overlay has no display cycles of its own; each frame fetch also reads the overlay word for the same 16 pixels. The ACRTC draws it through a character (CHR) screen that the board routes to the overlay memory and never displays; setup gives it the frame's coordinates. Switching to the overlay sets the chip to 1 bit a pixel, and back to 8, after waiting, so that queued commands run at the depth they were sent for.

11. Text

cz88gfx draws text a character at a time with the ACRTC's PTN command: each glyph is unpacked into the pattern RAM and PTN draws the character cell at the pen. That brings scaling (the pattern zoom, 1-16 times), turning (PTN's direction, in 45-degree steps) and the drawing modes with it. Text goes on whichever plane is being drawn, frame or overlay, in CL1.

11.1 Fonts

A font is chosen by naming it, and naming it is also what links it in, so a program carries only the fonts it uses. Both cover characters 20H-7EH; others draw as a blank.

Assembly	C	Cell	Glyphs	Bytes
CZF8	CZ_F8	8x8	5x7, no true descenders	480
CZF11	CZ_F11	9x11	7x10: capitals 8 rows, 2 descender rows	860



Figure 6. CZF8, the 8x8 font.

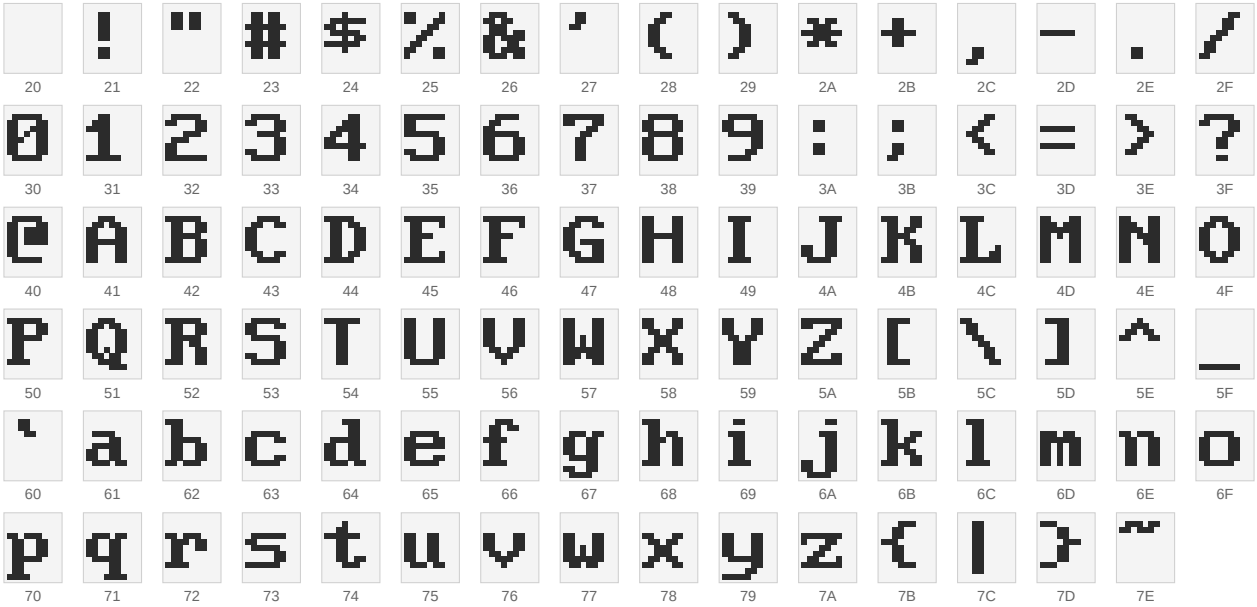


Figure 7. CZF11, the 9x11 font.

11.2 Drawing text

Assembly	C	Does
`GFONT f`	`cz_font(f)`	draw text in font f (the macro clobbers HL)
`GTPOS x,y`	`cz_tpos(x, y)`	put the pen at x,y: the bottom-left corner of the next character's cell; also the start of the line
`GTATTR s,d,f`	`cz_tattr(s, d, f)`	scale 1-16, direction 0-7, flags 0 or opaque (the macro clobbers BC)
`GTEXT "text"`	`cz_text("text")`	draw the text at the pen. The macro places it inline (clobbers HL, B); the function takes a NUL-terminated string, 255 characters at most
`GTSTR (HL, B)`	`cz_tstr(s, n)`	draw n characters from s
`GTCHR c`	`cz_tchr(c)`	draw one character
`GTNL`	`cz_tnl()`	a new line: the start of the line moves one line on, and the pen goes there

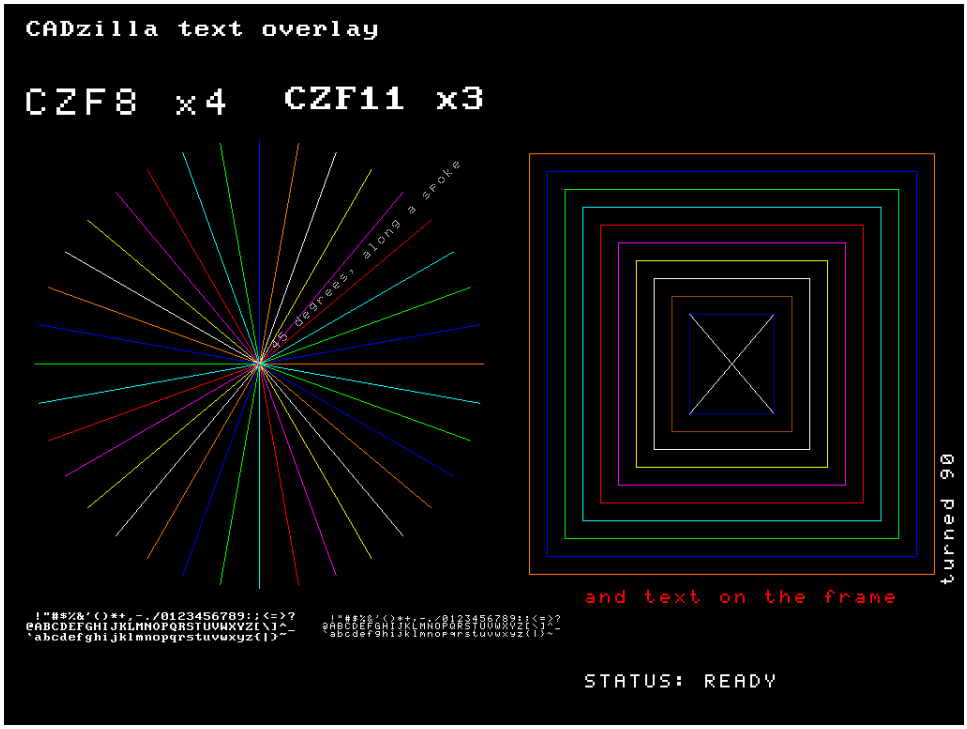
Assembly (z80asm)

```
        GOVL                    ;on the overlay
        GCOLOR  OVSET
        GFONT   CZF11
        GTATTR  2,0,0           ;scale 2, +X
        GTPOS   24,730
        GTEXT   "CADzilla text overlay"
        GTNL                    ;the next line
        GFRAME
```

C (zcc)

```
cz_ovl();                /* on the overlay */
cz_color(CZ_OVSET);
cz_font(CZ_F11);
cz_tattr(2, 0, 0);       /* scale 2, +X */
cz_tpos(24, 730);
cz_text("CADzilla text overlay");
cz_tnl();                /* the next line */
cz_frame();
```

- After each character the pen moves one cell width times the scale along the direction; a string is characters x cell width x scale long. A new line moves one cell height times the scale, 90 degrees clockwise from the direction (down, for ordinary text).
- Both fonts have a blank top row and blank right-hand columns, so text set at the cell pitch doesn't touch.
- **Transparent** text (the default) leaves the picture alone in a glyph's 0 bits. **Opaque** text (`TX0PAQ` / `CZ_TX0PAQ`) draws the whole cell, the 0 bits in CL0: for text rewritten in place, such as a status line. On the overlay, set CL0 to `0VCLR`.
- The color mode is forced only while a string is drawn; the operation and area modes apply as usual.
- A string leaves the pattern RAM as setup does, and moves the CP: load your own pattern, and move, afterwards.
- Choose a font first: until then, text draws nothing (in assembly, the calls return C).



Demo screen 10: screen 2 on the frame, and on the overlay a 9x11 title at scale 2, both fonts scaled up, both character sets (three lines each), text turned 90 and 45 degrees, and an opaque status line rewritten in place. The red line is drawn on the frame itself.

Size: the string-drawing core (CZTXT, 444 bytes) is all a program that only draws strings needs; the pen, attribute, character and newline calls are a separate module (CZTXL). An assembly program that wants to stay small can store the pen, scale and direction into the text state CZTST itself, at offsets +0/+2 (pen X, Y), +8 (scale), +9 (direction) and +10 (flags), and call CZTFNT and CZTSTR directly.

12. Reading back

Assembly	C	Returns
GGETCP	cz_getcp(&x, &y)	the CP: in HL and DE, or into x and y. It also clears ARD.
GRDOT x,y	cz_rdot(x, y)	the pixel at x,y: its palette index (in A, NC). Off the screen: C with A = 0; in C, -1

Both read through the ACRTC's read FIFO, after everything sent before them has been drawn. Reading a pixel reads the frame, whichever plane is being drawn.

13. Library versions

cz88gfx is released in numbered versions, and each program builds against the release it pins. A new cz88gfx version therefore reaches a program only when the program moves to it.

Where	What
`CZ88GFX.INC: CZ88VER` `cz88gfx.h: CZ88GFX_VERSION`	the version: `0101H` / `0x0101` is 1.1, major in the high byte, minor in the low. A program can check it at assembly or compile time.
`software/z88dk/cz88gfx/releases/<v>/`	the releases: `CZ88GFX.LIB`, `CZ88GFX.INC` and (1.1 on) `cz88gfx.h`, tracked, read-only and never changed
`software/<toolchain>/<project>/cz88gfx.version`	the pin: one line, e.g. `1.1`, or `dev` for the library's own current build

Release	What changed
1.0	czgfx 1.0 ported to z80asm
1.1	the C bindings: `cz88gfx.h` and the `CZC*` modules. The graphics modules are byte for byte 1.0's. A C program needs 1.1 or later.

13.1 Moving a program to another release

```
echo 1.1 > software/z88dk/myprog/cz88gfx.version
tools/z88dk/z88build.sh software/z88dk/myprog        (or zccbuild.sh)
```

The build puts that release on the include path and links its library; nothing needs mounting or copying. A program's `.COM` keeps the library code it was built with until it is rebuilt. A pin of `dev` builds against the library's own `src/` and `build/`, and warns if that build is older than its sources.

13.2 Making a release

Raise `CZ88VER` in `CZ88GFX.INC` and `CZ88GFX_VERSION` in `cz88gfx.h` together, rebuild the library (Appendix E), then run `software/z88dk/cz88gfx/tools/release.sh`. It copies the library and both headers into `releases/<major>.<minor>/`, read-only, and refuses to overwrite an existing release, to release a library older than its sources, or to release when the two version numbers disagree. Add the new release to git.

Appendix A. Macro reference (assembly)

Every argument is required. The register form of each macro, for computed arguments, is in Appendix B.

Macro	Does
`GINIT mode`	set up the board for CZM640, CZM800 or CZM1K; clear. Clobbers all
`GCLS c`	clear the screen to c
`GCOLOR c`	drawing color (CL1)
`GBCOL c`	background color CL0
`GCCMP c`	compare color for OMEQ/OMNE
`GPAL i,r,g,b`	set a palette entry. Clobbers BC, E
`GWAIT`	wait for the drawing to finish: A = status, Z = OK
`GABORT`	abort: flush the FIFOs, clear CER
`GMOVE x,y GRMOVE dx,dy`	move the CP
`GLINE x,y GRLINE dx,dy`	line from the CP (end not drawn)
`GRECT x,y GRRECT dx,dy`	rectangle outline from the CP
`GFRECT x,y GRFRCT dx,dy`	filled rectangle from the CP
`GDOT x,y`	one pixel
`GSEG x1,y1,x2,y2`	a line
`GBOX x1,y1,x2,y2`	a rectangle outline
`GFBOX x1,y1,x2,y2,c`	a filled rectangle in color c
`GPLINE ((pts)) GPOLY ((pts))`	polyline / polygon, points inline. Clobbers HL
`GPTS name,(pts)`	define a point table
`GPLT tbl GPLG tbl`	polyline / polygon from a table. HL ends past it
`GCIRC x,y,r`	circle
`GELLIP x,y,rx,ry`	ellipse
`GARC xc,yc,xe,ye,dir GRARC`	circular arc from the CP. Clobbers HL
`GEARC rx,ry,xc,yc,xe,ye,dir GREARC`	elliptical arc from the CP. Clobbers HL
`GMODE m`	the whole drawing mode
`GOPM op GCMODE cm GAMODE am`	one field of the drawing mode
`GAREA x1,y1,x2,y2 GAFULL`	the area / the whole screen
`GGETCP`	read the CP: HL, DE. Clears ARD
`GRDOT x,y`	read a pixel: A; C if off the screen
`GPAT name,(w...) GTILE name,(c...)`	define a pattern / color tile
`GLDPAT tbl,pra`	load the pattern RAM from word pra. Clobbers HL
`GPSCAN x0,y0,x1,y1,zx,zy`	pattern window and zoom. Clobbers HL
`GPPTR x,y`	pattern pointer. Clobbers HL

Macro	Does
`GPDEF`	pattern RAM, window and pointer as after setup
`GLSTYL pat,zoom`	line style
`GPTN x,y,w,h,dir`	draw the pattern RAM
`GPAINT x,y,e`	paint out to edge color e
`GFLOOD x,y,c`	recolor the region of color c
`GCOPY x1,y1,x2,y2,xd,yd,t`	copy a block. Clobbers HL
`GOVON GOVOFF`	show / hide the overlay
`GOVPAL r,g,b`	overlay color. Clobbers BC, E
`GOVL GFRAME`	draw on the overlay / the frame
`GOVCLS`	clear the overlay
`GFONT f`	choose a font, CZF8 or CZF11. Clobbers HL
`GTPOS x,y`	text pen
`GTATTR s,d,f`	text scale, direction, flags (0 or TXOPAQ). Clobbers BC
`GTEXT "text"`	draw inline text. Clobbers HL, B
`GTSTR GTCHR c`	draw B characters at HL / one character
`GTNL`	new line

Appendix B. Modules and routines

The routines behind the macros, by module, with each module's size in cz88gfx 1.1. Assembly calls them by name; a program links only the modules it uses, plus the ones those call.

Module	Bytes	Routines and data
CZCORE	167	CZWREG (B = register, HL = value), CZPUTW (HL to the FIFO), CZWAIT, CZXY / CZXYM (A = opcode, HL, DE), CZCMD (A = opcode, with the mode), CZWPR (A = drawing register, HL), CZSEND (HL = table, BC = words), CZRTAB, CZLDW, CZABT; data CZDM
CZINIT	566	CZINIT (HL = descriptor), CZCLS (A), CZOCLS, CZPDEF; data CZM640 CZM800 CZM1K, CZXMAX, CZYMAX, CZMREG (the MODE register), CZDPB
CZPAL	129	CZPALD (default palette), CZPALS (A = index, B/C/E = R/G/B)
CZDRAW	66	CZCOL (A), CZMOVE, CZLINE, CZRECT, CZFRCT, CZDOT, CZRMOV, CZRLIN, CZRRCT, CZRFRC (HL = X, DE = Y)
CZPOLY	49	CZPLL, CZPLG (HL = point table)
CZCURV	192	CZCRCL (HL = r), CZELPS (HL = rx, DE = ry), CZARC (A = OPAARC/OPRARC/OPAEAR/OPREAR + DIRCW, HL = table)
CZFILL	51	CZPANT (A = edge), CZFLOD (A = region), CZGCPY (A = transform, +10H relative; HL = table Xs,Ys,DX,DY)
CZPAT	149	CZLPAT (A = word, HL = table), CZLSTY (HL = style, A = zoom), CZPTN (HL = size, A = direction), CZPSCN (HL = 6-byte table)
CZMODE	246	CZOPM, CZCMOD, CZAMOD (A), CZCOL0, CZCCMP (A), CZAMIN, CZAMAX (HL, DE), CZAFUL, CZGCP, CZRDOT
CZOVL	78	CZOVON, CZOVOF, CZOPAL (B/C/E = R/G/B), CZOVL, CZFRM
CZTXT	444	CZTFNT (HL = font), CZTSTR (HL = text, B = length); data CZTST
CZTXL	149	CZTPOS (HL, DE), CZTATR (A = scale, B = direction, C = flags), CZTCHR (A), CZTNL
CZFNT8	480	data CZF8, the 8x8 font
CZFN11	860	data CZF11, the 9x11 font

A program that only sets up the screen and draws a line is 953 bytes of library (CZCORE, CZINIT, CZPAL and CZDRAW). The C bindings' modules, CZC*, add 660 bytes in all, of which a program links only those it uses (Appendix C).

Appendix C. C function reference

Every function in `cz88gfx.h`, with the macro or routine it stands for (whose description it shares), its calling convention and the stub module that defines it. This table is generated from the header and the stubs themselves.

- **fastcall**: one argument, passed in HL.
- **callee**: the arguments pushed left to right, popped by the function; no clean-up at the call site.
- **alias**: the library routine itself under a C name (a `DEFC`), with no stub code at all.

Prototype	Assembly	Convention	Module
`void cz_init(const void *mode)`	`GINIT`	fastcall, alias	`CZCINIT`
`void cz_cls(unsigned char c)`	`GCLS`	fastcall	`CZCINIT`
`void cz_pal(unsigned char i, unsigned char r, unsigned char g, unsigned char b)`	`GPAL`	callee	`CZCPAL`
`void cz_paldef(void)`	`CALL CZPALD`	no arguments, alias	`CZCPAL`
`void cz_color(unsigned char c)`	`GCOLOR`	fastcall	`CZCDRAW`
`void cz_move(int x, int y)`	`GMOVE`	callee	`CZCDRAW`
`void cz_line(int x, int y)`	`GLINE`	callee	`CZCDRAW`
`void cz_rect(int x, int y)`	`GRECT`	callee	`CZCDRAW`
`void cz_frect(int x, int y)`	`GFRECT`	callee	`CZCDRAW`
`void cz_dot(int x, int y)`	`GDOT`	callee	`CZCDRAW`
`void cz_rmove(int dx, int dy)`	`GRMOVE`	callee	`CZCDRAW`
`void cz_rline(int dx, int dy)`	`GRLINE`	callee	`CZCDRAW`
`void cz_rrect(int dx, int dy)`	`GRRECT`	callee	`CZCDRAW`
`void cz_rfrect(int dx, int dy)`	`GRFRCT`	callee	`CZCDRAW`
`void cz_seg(int x1, int y1, int x2, int y2)`	`GSEG`	callee	`CZCDRAW`
`void cz_box(int x1, int y1, int x2, int y2)`	`GBOX`	callee	`CZCDRAW`
`void cz_fbox(int x1, int y1, int x2, int y2, unsigned char c)`	`GFBOX`	callee	`CZCDRAW`
`void cz_polyline(const int *t)`	`GPLT`	fastcall, alias	`CZCPOLY`
`void cz_polygon(const int *t)`	`GPLG`	fastcall, alias	`CZCPOLY`
`unsigned char cz_wait(void)`	`GWAIT`	no arguments	`CZCCORE`
`void cz_abort(void)`	`GABORT`	no arguments, alias	`CZCCORE`
`void cz_circle(int x, int y, unsigned int r)`	`GCIRC`	callee	`CZCCURV`
`void cz_circle_cp(unsigned int r)`	`CALL CZCRCL`	fastcall, alias	`CZCCURV`
`void cz_ellipse(int x, int y, unsigned int rx, unsigned int ry)`	`GELLIP`	callee	`CZCCURV`
`void cz_ellipse_cp(unsigned int rx, unsigned int ry)`	`CALL CZELPS`	callee	`CZCCURV`
`void cz_arc(int xc, int yc, int xe, int ye, unsigned char dir)`	`GARC`	callee	`CZCCURV`
`void cz_rarc(int xc, int yc, int xe, int ye, unsigned char dir)`	`GRARC`	callee	`CZCCURV`

Prototype	Assembly	Convention	Module
`void cz_earc_sq(unsigned int a, unsigned int b, int xc, int yc, int xe, int ye, unsigned char dir)`	GEARC	callee	CZCCURV
`void cz_rearc_sq(unsigned int a, unsigned int b, int xc, int yc, int xe, int ye, unsigned char dir)`	GREARC	callee	CZCCURV
`void cz_paint(int x, int y, unsigned char edge)`	GPAINT	callee	CZCFILL
`void cz_flood(int x, int y, unsigned char c)`	GFLOOD	callee	CZCFILL
`void cz_paint_cp(unsigned char edge)`	CALL CZPANT	fastcall	CZCFILL
`void cz_flood_cp(unsigned char c)`	CALL CZFLOD	fastcall	CZCFILL
`void cz_copy(int x1, int y1, int x2, int y2, int xd, int yd, unsigned char t)`	GCOPY	callee	CZCFILL
`void cz_setmode(unsigned char m)`	GMODE	fastcall	CZCCORE
`void cz_opm(unsigned char m)`	GOPM	fastcall	CZCMODE
`void cz_cmode(unsigned char m)`	GCMODE	fastcall	CZCMODE
`void cz_amode(unsigned char m)`	GAMODE	fastcall	CZCMODE
`void cz_bcol(unsigned char c)`	GBCOL	fastcall	CZCMODE
`void cz_ccmp(unsigned char c)`	GCCMP	fastcall	CZCMODE
`void cz_area(int x1, int y1, int x2, int y2)`	GAREA	callee	CZCMODE
`void cz_afull(void)`	GAFULL	no arguments, alias	CZCMODE
`void cz_getcp(int *x, int *y)`	GGETCP	callee	CZCMODE
`int cz_rdot(int x, int y)`	GRDOT	callee	CZCMODE
`void cz_ldpat(const unsigned int *t, unsigned char pra)`	GLDPAT	callee	CZCPAT
`void cz_pscan(unsigned char x0, unsigned char y0, unsigned char x1, unsigned char y1, unsigned char zx, unsigned char zy)`	GPSCAN	callee	CZCPAT
`void cz_pptr(unsigned char x, unsigned char y)`	GPPTR	callee	CZCPAT
`void cz_pdef(void)`	GPDEF	no arguments, alias	CZCINIT
`void cz_lstyle(unsigned int pat, unsigned char zoom)`	GLSTYL	callee	CZCPAT
`void cz_ptn(int x, int y, unsigned int w, unsigned int h, unsigned char dir)`	GPTN	callee	CZCPAT
`void cz_ptn_cp(unsigned int w, unsigned int h, unsigned char dir)`	CALL CZPTN	callee	CZCPAT
`void cz_ovon(void)`	GOVON	no arguments, alias	CZCOVL
`void cz_ovoff(void)`	GOVOFF	no arguments, alias	CZCOVL
`void cz_ovl(void)`	GOVL	no arguments, alias	CZCOVL
`void cz_frame(void)`	GFRAME	no arguments, alias	CZCOVL
`void cz_ovcls(void)`	GOVCLS	no arguments, alias	CZCINIT
`void cz_ovpal(unsigned char r, unsigned char g, unsigned char b)`	GOVPAL	callee	CZCOVL
`void cz_font(const void *f)`	GFONT	fastcall, alias	CZCTXT
`void cz_tpos(int x, int y)`	GTPOS	callee	CZCTXL
`void cz_tattr(unsigned char scale, unsigned char dir, unsigned char flags)`	GTATTR	callee	CZCTXL

Prototype	Assembly	Convention	Module
`void cz_text(const char *s)`	`GTEXT`	fastcall	`CZCTXT`
`void cz_tstr(const char *s, unsigned char n)`	`GTSTR`	callee	`CZCTXT`
`void cz_tchr(char c)`	`GTCHR`	fastcall	`CZCTXL`
`void cz_tnl(void)`	`GTNL`	no arguments, alias	`CZCTXL`
`void cz_putw(unsigned int w)`	`CALL CZPUTW`	fastcall, alias	`CZCCORE`
`void cz_cmd(unsigned char op)`	`CALL CZCMD`	fastcall	`CZCCORE`
`void cz_wpr(unsigned char r, unsigned int v)`	`CALL CZWPR`	callee	`CZCCORE`
`void cz_wreg(unsigned char r, unsigned int v)`	`CALL CZWREG`	callee	`CZCCORE`

Data, and the macros the header defines:

Declaration	What it is
`unsigned char cz_m640[], cz_m800[], cz_m1k[]`	the mode descriptors; `CZ_M640`, `CZ_M800`, `CZ_M1K` name them
`unsigned int cz_xmax, cz_ymax`	the current mode's largest x and y (read only)
`unsigned char cz_f8[], cz_f11[]`	the fonts; `CZ_F8`, `CZ_F11` name them, and naming one links it in
`cz_earc(rx, ry, xc, yc, xe, ye, dir)` `cz_rearc(...)`	macros: `cz_earc_sq` / `cz_rearc_sq` with rx, ry squared
`CZ_TILE(c)`	a color tile word: c in both bytes

C equivalents of the register forms: `cz_circle_cp`, `cz_ellipse_cp`, `cz_paint_cp`, `cz_flood_cp` and `cz_ptn_cp` work at the CP. `cz_putw`, `cz_cmd`, `cz_wpr` and `cz_wreg` send to the ACRTC directly, for the commands the library has no call for (Appendix G).

Appendix D. Equates and constants

Besides the macros and functions, the headers define:

Group	Assembly (CZ88GFX.INC)	C (cz88gfx.h, cadzilla.h)
Ports	CZBASE (70H), ACRSR, MODEREG, ACRDAT, DACADR, DACPAL, DACOVL	CZ_BASE, CZ_ACRTC_SR / _AR, CZ_MODE, CZ_ACRTC_DATA, CZ_DAC_ADDR, CZ_DAC_PAL, CZ_DAC_OVL; CZ_PORT_... numbers
Status bits	SRWFE 01H, SRWFR 02H, SRRFR 04H, SRCED 20H, SRARD 40H, SRCER 80H	CZ_SR_WFE ... CZ_SR_CER (also RFF, LPD)
MODE bits	MDOLEN 08H, MDOLSL 10H	CZ_MODE_HSPOL, VSPOL, AMODE, OLEN, OLSEL
Hues	ORANGE BLUE GREEN CYAN RED MAGENTA YELLOW WHITE (0-224); BRIGHT 31	CZ_ORANGE ... CZ_WHITE; CZ_BRIGHT
Operation modes	OMREP OMOR OMAND OMEOR OMEQ OMNE OMLT OMGT (0-7)	CZ_OM_REP ... CZ_OM_GT
Color modes	CMSEL 00H, CMCL1 08H, CMCL0 10H, CMPRAM 18H	CZ_CM_SEL, _CL1, _CL0, _PRAM
Area modes	AMNONE 00H, AMSTOP 20H, AMCLIP 40H, AMCLPD 60H, AMSTPI 0A0H, AMHOLE 0C0H, AMHOLD 0E0H	CZ_AM_NONE, _STOP, _CLIP, _CLPD, _STPI, _HOLE, _HOLD
Copy transforms	CPNORM CPMIRY CPMIRX CPR180 CPTRAN CPR270 CPR90 CPATRN (0-7)	CZ_CP_NORM ... CZ_CP_ATRN
Directions	DIRCCW 0, DIRCW 1; PTSLNT 8	CZ_DIR_CCW, CZ_DIR_CW; CZ_PTSLNT
Overlay, text	OVSET 0FFH, OVCLR 0; TXOPAQ 1	CZ_OVSET, CZ_OVCLR; CZ_TXOPAQ
Mode descriptors, fonts	CZM640 CZM800 CZM1K; CZF8 CZF11	CZ_M640 CZ_M800 CZ_M1K; CZ_F8 CZ_F11
Opcodes	OPMOVE OPRMOV OPLINE OPRLIN OPRECT OPRRCT OPAPLL OPAPLG OPCRCL OPELPS OPAARC OPRARC OPAEAR OPREAR OPFRCT OPRFRC OPPANT OPDOT OPPTN OPAGCP OPRGCP OPRD (command high bytes); CMDDOT	(none: use the values with cz_cmd)
Version	CZ88VER (0101H for 1.1)	CZ88GFX_VERSION (0x0101)

Appendix E. Building and extending the library

Everything runs on the host:

```
software/z88dk/cz88gfx/tools/build.sh
```

- It assembles every module in `src/` (8080 code, `-m8080`) into `build/CZ88GFX.LIB`, with listings, and lints the sources for `JP/CP`.
- It links `CZ88CHK.Z88` against the new library for the 8080, 8085 and Z80. CZ88CHK expands every macro, references every entry point and makes every register-form call.
- It compiles `czcchk.c` with zcc for each CPU's C library and links it against the new library. It calls every C function and names every constant.
- So a broken macro, a wrong prototype or a missing PUBLIC fails the library build, not the first program. The build is good when it ends with six `check: ... links` lines and the library size.

E.1 Adding to the library

- **A routine:** make it PUBLIC in its module, add it to the header's EXTERN block, add a macro if it helps, and exercise it in `CZ88CHK.Z88`.
- **Its C function:** a stub in the matching `CZC*.Z88` module, or a `DEFC` alias when the routine needs nothing but HL; its prototype in `cz88gfx.h`; a call in `czcchk.c`.
- **A module:** create `src/CZxxxx.Z88` (INCLUDE "CZ88GFX.INC", `SECTION code_user`, its PUBLICs) and add its names to the header's EXTERN block; give it C bindings in a new `src/CZCxxxx.Z88` of its own, so a C program that doesn't use it doesn't link it. `build.sh` picks up every module; z80asm has no module order to keep.
- **A font:** draw its glyphs in `tools/mkfont.py`, which writes the font modules.
- Try a change in a program pinned to `dev` (z88demo, or cgfxdemo for C), checking the pictures as well as the build, then release it (chapter 13).

E.2 How the C stubs work

- A one-argument (fastcall) stub moves the argument from HL to where the routine wants it, e.g. `MOV A,L` / `JMP CZCOL`.
- A stub whose arguments fit the registers pops them straight in: the return address, then the arguments last first, then the return address back, and a jump to the routine (7-11 bytes).
- A longer call uses `CZCPOP` (module `CZCARG`), which moves C argument words into the static block `CZCAV` in their declared order: often the very parameter table the routine takes, as for the arcs and the copy. Like the library's own state, this makes the library non-reentrant: don't draw from an interrupt handler.
- The stubs are 8080 code and in `SECTION code_user`, the section zcc's CP/M startup places with the program. They use only AF, BC, DE and HL, which sccz80 lets a function change.

Appendix F. Porting from czgfx

An M80 program for czgfx ports to cz88gfx's assembly side line for line (z88demo is gfxdemo ported this way); for C, see Appendix C.

czgfx (M80)	cz88gfx (z80asm)
INCLUDE B:CZGFX.INC / CSEG / END START	INCLUDE "CZ88GFX.INC" / ORG 0100H / (no END)
CALL CZLDW##, LHLD CZXMAX##	CALL CZLDW, LHLD CZXMAX
GINIT	GINIT CZM640 (always name the mode)
GCLS, GMODE	GCLS 0, GMODE 0
GCOLOR (A set), and the like	CALL CZCOL, CZCOL0, CZCCMP, CZOPM, CZCMOD, CZAMOD, CZTCHR
GMOVE (HL, DE set), and the like	CALL CZMOVE, CZLINE, CZRECT, CZFRCT, CZDOT, CZRDOT, CZTPOS
GMOVE ,420	LXI D,420 / CALL CZMOVE
GCIRC / GCIRC ,,7	CALL CZCRCL / LXI H,7 / CALL CZCRCL
GTATTR 2 / GTATTR 1,2	GTATTR 2,0,0 / GTATTR 1,2,0
GLDPAT t / GLSTYL p / GPTN x,y,w,h	GLDPAT t,0 / GLSTYL p,0 / GPTN x,y,w,h,0
GPSCAN x0,y0,x1,y1	GPSCAN x0,y0,x1,y1,0,0
GARC xc,yc,xe,ye	GARC xc,yc,xe,ye,DIRCCW
GCOPY x1,y1,x2,y2,xd,yd	GCOPY x1,y1,x2,y2,xd,yd,CPNORM
GPOLY <x0,y0,...>	GPOLY ((x0,y0,...))
GPTS n,<...>	GPTS n,(...)
GTEXT <'text'>	GTEXT "text"
GCOLOR <(N MOD 8)*32>	GCOLOR 0+(N % 8)*32
N SET N+1	N DEFL (N+1)
IRP V,<a,b> ... ENDM / REPT n ... ENDM	REPTI V,a,b ... ENDR / REPT n ... ENDR
DB 'text',CR,LF,'$'	DB "text",CR,LF,'$'

Appendix G. Limits

- No calls for the ACRTC's word-address block commands (CPY, SCPY, DRD, DWT, DMOD), WT and MOD, or the read-backs other than the CP and a pixel. The copy covers copying on the screen. For the rest, send the command yourself: `CZCMD` and `CZPUTW` in assembly, `cz_cmd` and `cz_putw` in C.
- PAINT's four-seed limit on the real chip (9.1).
- It is 8080 code everywhere: a Z80 program runs the same code an 8080 one does.
- The C bindings are for sccz80, the compiler `zccbuild.sh` uses; zcc's `-compiler=sdcc` (Z80 only) is untested with them.
- The library is assembled for I/O base 70H; `-DCZ_BASE` moves only `<cadzilla.h>`'s ports.
- Only the overlay's lower half (`MDOLSL` = 0) is used, since the screen starts at frame word 0.
- Switching the ACRTC's pixel depth while the display runs (the overlay switches) is fine in altairsim, but has not yet been checked on the real board.